

Linguagens Formais e Autômatos

Prof. Yandre Costa

5 de julho de 2026

Site do Professor: <http://www.din.uem.br/yandre>

1 Teoria da Computação

→ Ciência da Computação

- Ênfase teórica: ideias fundamentais e modelos computacionais.
- Ênfase prática: projeto de sistemas computacionais.

→ A teoria da computação possui duas grandes vertentes.

1. Linguagens Formais e Autômatos.
2. Máquinas Universais e Computabilidade.

→ O que é Teoria da Computação?

- Estudos teóricos acerca da capacidade de resolução de problemas das máquinas.
- Estudo de modelos formais que:
 - * Caracterizam em nível conceitual: programas, máquinas e enfim, a computação.
 - * Especificam o que é computável ou não.
 - * Ajudam na especificação de linguagens artificiais entre outras aplicações.

→ Histórico da Computação

- Computar, do latim "computare", que significa calcular, avaliar, contar.

2 Conceitos Básicos

• Introdução

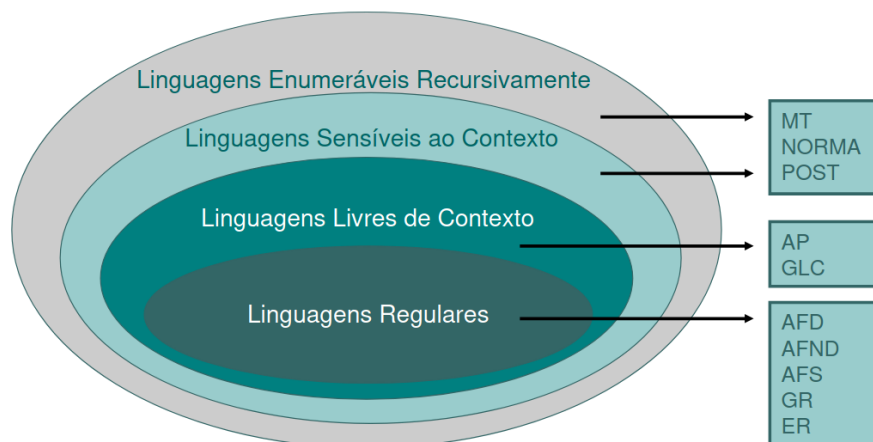
- Problema: definir um conjunto de cadeias de símbolos
- Exemplo: conjunto M dos números binários que têm 2 dígitos

$$M = \{00, 01, 10, 11\}$$

- Porém, se fosse o conjunto N de todas as combinações de dígitos binários, poderíamos tentar o seguinte:

$$N = \{0, 1, 00, 01, 10, 11, 000, \dots\}$$

- * Percebe-se que este conjunto é infinito
- Representação clara: humanos x computador
- Representação formal: computador
- Um objetivo de LFA é estudar uma maneira precisa e formal de descrever sequências de símbolos pertencentes a um determinado conjunto.
- Em especial conjuntos que não podem ser trivialmente enumerados.
- Os estudos iniciais foram em torno de Linguagens Naturais (LN).
- Algumas características de LN introduziram dificuldades no tratamento computacional das mesmas.
 - * LN é extensa, complexa, não tem sintaxe rígida e semântica bem determinada (rica em ambiguidade)
- As maneiras sistêmicas de descrever uma linguagem de programação são:
 - * Um método que permite construir programas sintaticamente corretos - geração (Gramática)
 - * Um método que permite verificar se um programa escrito está sintaticamente correto - reconhecimento (Autômatos)
- Hierarquia de Chomsky



- Pesquisas já demonstraram o uso destas técnicas em:
 - * Análise de linguagens de programação
 - Léxica
 - Sintática
 - * Modelos de sistemas biológicos
 - * Desenho de hardware
 - * Relacionamentos com linguagens Naturais
 - * Reconhecimento de padrões (em visão computacional por exemplo)

- Alfabeto
 - Conjunto de simbolos finito e não vazio
 - Normalmente denotado por Σ
 - Exemplos
 - * $\Sigma = \{a, b\}$
 - * $\Sigma = \{1, 2, 3\}$
 - * $\Sigma = \{00, 11\}$
- Simbolo ou letra
 - É todo elemento pertencente a um alfabeto
 - a é um simbolo de Σ
- Cadeia ou palavra
 - É uma concatenação de simbolos do alfabeto justapostos.
 - Assim, dado um alfabeto Σ e uma sequencia de simbolos $x = a_1a_2a_3 \dots a_n$, x é uma cadeia sobre Σ se, e somente se, $a_i \in \Sigma$ para $i = 1, 2, \dots, n$.
 - Comprimento de Cadeia ou Tamanho de Palavra
 - * É o numero de simbolos que compoem uma dada cadeia (ou palavra)
 - * O comprimento de uma cadeia x é denotado por $|x|$
 - * Então, a cadeia $x = a_1a_2a_3 \dots a_n$ tem seu comprimento $|x| = n$
 - * Cadeia nula ou palavra vazia: é o caso especial, ela é denotada por λ (ou ϵ) e tem o tamanho igual a zero
 - * Exercício: dado o alfabeto $\Sigma = a, b, c, de$, verifique se as cadeias a seguir são formadas sobre este alfabeto, e se for, verifique qual o comprimento
- Exponenciação de Alfabetos
 - Σ^k é o conjunto de todas as cadeias com tamanho k formadas sobre o alfabeto Σ
 - Exemplo: Considere $\Sigma = \{0, 1\}$
 - * $\Sigma^0 = \lambda$
 - * $\Sigma^1 = \{0, 1\}$
 - * $\Sigma^2 = \{00, 01, 10, 11\}$
 - * $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$
- Fechamento de um Alfabeto
 - Seja Σ um alfabeto, então, o fechamento de Σ descrito por Σ^* é definido como

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots \cup \Sigma^n \cup \dots$$
- Concatenação de cadeias
 - Dado o alfabeto Σ e as cadeias $x, y \in \Sigma^*$, a concatenação de x e y , indicada por xy , produz uma cadeia formada pelos simbolos de x seguidos pelos simbolos de y .

- Se $x = a_1a_2a_3 \dots a_n \in \Sigma^*$ e $y = b_1b_2b_3 \dots b_m \in \Sigma^*$ então:

$$xy = a_1a_2a_3 \dots a_nb_1b_2b_3 \dots b_m$$

- Exemplos:

$$\Sigma = \{a, b\}$$

$$x = abaa, y = ba, z = \lambda$$

$$xy = abaaba$$

$$yx = baabaa$$

$$yz = ba = zy = y$$

- A cadeia nula (λ) é o elemento neutro da concatenação
- Concatenação Sucessiva – Concatenação de uma palavra com ela mesma.
 - * Representada através de um expoente: w^n – Onde w é uma palavra e n indica o número de concatenações sucessivas.

- Dado um alfabeto Σ e $x, y \in \Sigma^*$, diz-se que:

- x é um prefixo de y , se e somente se, $\exists w \in \Sigma^*$ tal que $y = xw$
- x é um sufixo de y , se e somente se, $\exists w \in \Sigma^*$ tal que $y = wx$
- x é uma subpalavra de y , se e somente se, $\exists w, u \in \Sigma^*$ tal que $y = wxu$

- Linguagem

- Dado o alfabeto Σ , o conjunto de palavras L é uma linguagem sobre Σ se e somente se $L \subset \Sigma^*$.

- Operações sobre linguagens:

- * Considere L_1 e L_2 linguagens definidas sobre Σ

- União: $L_1 \cup L_2 = \{x | x \in L_1 \vee x \in L_2\}$
- Interseção: $L_1 \cap L_2 = \{x | x \in L_1 \wedge x \in L_2\}$
- Diferença: $L_1 - L_2 = \{x | x \in L_1 \wedge x \notin L_2\}$
- Concatenação: $L_1L_2 = \{x | x = yz, y \in L_1 \wedge z \in L_2\}$
- Complemento: $\bar{L}_1 = \{x | x \in \Sigma^* \wedge x \notin L_1\}$

- Comparando as definições:

- Linguagem Natural:

- * Uma palavra em português equivale a um símbolo
- * Uma sentença da língua portuguesa é uma cadeia composta de vários símbolos.

- Linguagem computacional

- * Cada programa escrito numa linguagem computacional corresponde a uma cadeia de símbolos que podem ser:
 - Identificadores;
 - Palavras reservadas;
 - Símbolos especiais e operadores;
 - Constantes numéricas.

3 Autômatos Finitos

- AFD – Modelo matemático para definição de linguagem
- Caráter reconhecedor
- Modelam também sistemas de estados finitos
- Exemplo clássico: problema HLCR (Hopcroft e Ullman)

- Problema: um homem quer atravessar um rio levando consigo um lobo, uma cabra e um repolho, e no bote, só cabem ele e mais um dos outros três.
- Exemplos de possíveis estados do sistema:
 - $\langle \text{HLCR-0} \rangle$ – todos na margem esquerda
 - $\langle \text{L-HCR} \rangle$ – lobo na margem esquerda, cabra e repolho na direita
- Entradas do sistema:
 - h – Homem atravessa o rio sozinho
 - l – Homem atravessa o rio com o lobo
 - c – Homem atravessa o rio com a cabra
 - r – Homem atravessa o rio com o repolho
- Objetivo: $\langle \text{HLCR-0} \rangle \Rightarrow \langle \text{0-HLCR} \rangle$
- Representação por diagrama:
 - Círculos representam estados
 - Arcos representam ação ou transição (de um estado para o outro)
 - O estado final é marcado por um círculo duplo
 - As respostas para o problema são as sequências de ações que levam do estado inicial para o final

3.1 Autômato Finito Determinístico

- Exemplo de sistema que pode ser representado desta forma:
 - Forno de micro-ondas
 - * Entradas: porta aberta ou fechada, comandos fornecidos pelo cozinheiro através do painel, sinal do "timer" que expira
 - * Estados: aberto, esperando por comandos, cozinhando, desligado
- Definições Básicas
 - Um AFD A define uma linguagem $L(A)$ sobre um alfabeto Σ
 - Caráter reconhecedor, ao contrário das gramáticas estudadas que tinham caráter gerador

- * Dada uma cadeia x , ela pertence a $L(A)$
- Uma abstração de um AFD
 - * Uma cabeça de leitura extrai sequencialmente o conteúdo de uma fita (string)
 - * Uma luz de aceitação que acende somente se a cadeia pertencer a linguagem representada pela AFD
 - * Exemplos de cadeias aceitas em HLCR
 - chrcihc, ccchllrclllhccc, ...
- Definição Matemática de um AFD
 - * Uma AFD é uma quintupla $\langle \Sigma, S, S_0, \delta, F \rangle$, onde:
 - Σ é o alfabeto de Entradas
 - S é um conjunto finito não vazio de estados
 - S_0 é o estado inicial, $S_0 \in S$
 - δ é a função de transição de estados, definida $\delta : S \times \Sigma \rightarrow S$
 - F é o conjunto de estados finais, $F \subseteq S$
 - * Uma cadeia x para ser aceito, deve levar do estado S_0 para algum estado pertencente a F
 - * A função δ determina como são as transições de estados. Ela leva um par $\langle s, a \rangle$ onde s é um estado e a uma letra do alfabeto num estado s'
 - $\delta(s, a) = s'$
 - * Então, dado um AFD $A = \langle \Sigma, S, S_0, \delta, F \rangle$ e a cadeia $x = a_1 a_2 a_3 \dots a_n \in \Sigma^*$, o autômato parte de S_0 . Ao processar a_1 passa para o estado $\delta(S_0, a_1)$, ao processar a_2 passa para o estado $\delta(S_0, a_2)$, e assim por diante até processar a_n . Nesse ponto, o autômato estará num estado R qualquer. Se $R \in F$, então a cadeia $x \in L(A)$.
 - * Finito: número de estados envolvidos no sistema é finito
 - * Determinístico: Estabelece que para uma cadeia $x \in L(A)$, só existe uma única sequência de estados no AFD A para processá-la.

• Exercícios

- Dados os seguintes autômatos:
- Identifique a linguagem associada a cada um
 - R: Autômato 1: $L = \{a^n b c^m \vee a^n c b^m | n \geq 0, m \geq 0\}$
 - Autômato 2: $L = \{a a^n b b^m | n \geq 0, m \geq 0\}$
 - Autômato 3: $L = \{a..z A..Z(a..z A..Z 0..9)^n | n \geq 0\}$
- Descreva as funções de transição de cada um (exceto para o terceiro diagrama)

R: Autômato 1:

$$\delta(S_0, a) = S_0; \delta(S_0, b) = S_1; \delta(S_1, c) = S_1$$

OU

$$\delta(S_0, a) = S_0; \delta(S_0, c) = S_2; \delta(S_2, b) = S_2$$

Autômato 2:

$$\delta(S_0, a) = S_1; \delta(S_1, a) = S_1; \delta(S_1, b) = S_2; \delta(S_2, b) = S_2$$

– Exemplo:

- * Modelagem de uma "vending machine" que aceita moedas de 5, 10 e 25 centavos. O preço do produto que ela entrega é 30 centavos.
- * Partindo do estado inicial (0 centavos), deveremos reconhecer sequencias que nos levem a estados finais (valor inserido ≥ 30 centavos)
- * Podemos chamar o autômato de V
- * Assim:

- $V = \langle \Sigma, S, S_0, \delta, F \rangle$ onde:
- $\Sigma = \{5, 10, 25\}$ – Cada um destes símbolos (ou letras) representa uma ação.
- $S = \{ \langle 0c \rangle, \langle 5c \rangle, \langle 10c \rangle, \langle 15c \rangle, \langle 20c \rangle, \langle 25c \rangle, \langle 30c \rangle, \langle 35c \rangle, \langle 40c \rangle, \langle 45c \rangle, \langle 50c \rangle \}$ – Cada estado indica quanto foi depositado.
- $S_0 = \langle 0c \rangle$ – estado inicial.
- $F = \{ \langle 30c \rangle, \langle 35c \rangle, \langle 40c \rangle, \langle 45c \rangle, \langle 50c \rangle \}$ – estado onde a entrada é válida e o produto pode ser liberado.
- Delta é definido como:

$$\begin{aligned}
 \delta(\langle 0c \rangle, 5) &= \langle 5c \rangle \\
 \delta(\langle 0c \rangle, 10) &= \langle 10c \rangle \\
 \delta(\langle 0c \rangle, 25) &= \langle 25c \rangle \\
 \delta(\langle 5c \rangle, 5) &= \langle 10c \rangle \\
 \delta(\langle 5c \rangle, 10) &= \langle 15c \rangle \\
 \delta(\langle 5c \rangle, 25) &= \langle 30c \rangle \\
 \delta(\langle 10c \rangle, 5) &= \langle 15c \rangle \\
 &\dots
 \end{aligned}$$

- Tabela de transição de estados

δ	5	10	25
$\langle 0c \rangle$	$\langle 5c \rangle$	$\langle 10c \rangle$	$\langle 25c \rangle$
$\langle 5c \rangle$	$\langle 10c \rangle$	$\langle 15c \rangle$	$\langle 30c \rangle$
$\langle 10c \rangle$	$\langle 15c \rangle$	$\langle 20c \rangle$	$\langle 35c \rangle$
$\langle 15c \rangle$	$\langle 20c \rangle$	$\langle 25c \rangle$	$\langle 40c \rangle$
$\langle 20c \rangle$	$\langle 25c \rangle$	$\langle 30c \rangle$	$\langle 45c \rangle$
$\langle 25c \rangle$	$\langle 30c \rangle$	$\langle 35c \rangle$	$\langle 50c \rangle$
$\langle 30c \rangle$	-	-	-
$\langle 35c \rangle$	-	-	-
$\langle 40c \rangle$	-	-	-
$\langle 45c \rangle$	-	-	-
$\langle 50c \rangle$	-	-	-

A tabela de transição é boa para demonstrar transições grandes, e também para implementar em codigos, usando uma matriz para descrever a tabela de transição!!

Para descrevermos que a quantia de itens é a definição de um AFD, podemos dizer, por exemplo, que $L = \{x \in \Sigma^* \mid x|_a \text{ condição} \}$

3.2 Autômato Finito Não Determinístico – AFND

- Modelo matemático semelhante ao AFD
- Condições mais flexíveis
- Podem haver múltiplos caminhos para processar uma cadeia
- Definição formal
 - Uma AFD é uma quintupla $\langle \Sigma, S, S_0, \delta, F \rangle$, onde:
 - * Σ é o alfabeto de entradas
 - * S é um conjunto finito não vazio de estados
 - * S_0 é um conjunto não vazio de estados iniciais, $S_0 \subseteq S$
 - * δ é a função de transição de estados, definida $\delta : S \times \Sigma \rightarrow \rho(S)$
 - A função pode ativar todos os possíveis subconjuntos dos estados
 - * F é o conjunto de estados finais, $F \subseteq S$
 - Σ, S e F são os mesmos do AFD;
 - S_0 era um único estado em AFD. Em AFND é um conjunto com pelo menos um estado inicial
 - * Então em AFD, $S_0 \in S$, e em AFND, $S_0 \subseteq S$
 - Um AFND pode ter mais de um estado "ativo" (corrente) num instante;
 - Inicialmente, todo estado de S_0 são ativados;
- "Alternativas" para processar uma única cadeia;
- Se o processamento de uma cadeia não levar ao estado final por um caminho, deve-se tentar por outros caminhos (se houverem);
- Se algum dos caminhos possíveis levar a cadeia x a um estado final, então ela faz parte da linguagem definida pelo automato.
- A cadeia é rejeitada se a partir de nenhum estado inicial for possível atingir um estado final ao término do processamento;
- A função δ agora é definida por $S \times \Sigma \rightarrow \rho(S)$
- Exemplo da função $\delta(S, a) = R, T$
imagem
- Se o estado S estiver ativo e a letra a for processada, então tanto R quanto T passaram a estar ativos;
- Se seguirmos por R e não chegarmos a um estado final, podemos tentar por T
- Se alguma das alternativas levar a um estado final, a cadeia é reconhecida
- Se nenhuma alternativa levar a um estado final, a cadeia é rejeitada;
- Ao processar uma letra, todas as transições rotuladas com aquela letra, a partir de todos os estados ativos serão efetuadas
- Então, podemos novamente observar que podemos ter vários estados ativos ao mesmo tempo, ao contrário dos AFDs

3.3 Equivalência entre AFD e AFND

- A classe dos AFDs é equivalente à classe dos AFNDs. Assim, para todo AFD existe um AFND equivalente e vice-versa.
- Exemplo – $L(A) = \{w | w \in \{a, b\}^* \wedge w \text{ possui } aaa \text{ como sufixo}\}$
imagem
- Exercício
 - Construa um AFD e um AFND para a seguinte linguagem:
 - * $L(A) = \{w | w \in a, b^* \wedge w\}$ possui aa como subpalavra
imagem
- Transformação de AFND em AFD
 - Para todo AFND existe um AFD equivalente
 - Dado um AFND $A = \langle \Sigma, S, S_0, \delta, F \rangle$, define-se o AFD $A^d = \langle \Sigma, S^d, S_0^d, \delta^d, F^d \rangle$ equivalente da seguinte forma:
 - * $S^d = \rho(S)$
 - * $S_0^d = S_0$
 - * F^d : Todas as combinações de estados de S^d que possui como componente algum estado de F
 - * Seja $Q \in S^d$, $\delta^d(Q, a)$ vai levar à um estado que corresponde à combinação de todos os estados que podem ser alcançados ao processar o símbolo 'a' a partir de qualquer componente Q
 - Exemplo 1:
 - * Dado o AFND:
imagem
 - * Cria um estado para cada combinação possível de estados do AFND
 - * Estabelece-se como estado inicial a combinação correspondente ao conjunto de estados iniciais do AFND
 - * Estados finais: todos que possuem em sua combinação algum dos estados finais do AFND
 - * Criação das transições:
 - $\delta^d(Q, a)$ será igual ao estado que corresponda à combinação de todos os estados que formam o conjunto $\delta(Q, a)$
imagem
 - Durante a transformação de um AFND em um AFD podem ser criados estados que fiquem "isolados" do(s) estado(s) inicial(is)
 - Exercício: Transforme o AFND descrito a seguir em um AFD
imagem
- Minimização de AFD
 - Dois AFDs A e B são equivalentes, denotado por $A \equiv B$, se $L(A) = L(B)$

- Um automato minimo para uma linguagem regular L é um automato com o menor numero de estados possivel que aceita L
 - * Um AFD $A = \langle \Sigma, S_A, S_{0A}, \delta_A, F_A \rangle$ é dito minimo se para qualquer AFD $B = \langle \Sigma, S_B, S_{0B}, \delta_B, F_B \rangle$ tal que $A \equiv B$, temos que $|S_A| \leq |S_B|$
- Pré-requisitos para a minimização:
 - * O automato deve ser Deterministico
 - * Não pode ter estados inacessiveis a partir do estado inicial
 - * A função de transição deve ser total (todas as saídas devem ser previstas para todos os estados)
 - Para este ultimo requisito muitas vezes é necessario introduzir um estado S_{trash} para lançar as transições não previstas originalmente
- A estratégia consiste em fundir estados equivalentes num mesmo estado
 - * Dois estados são equivalentes se, ao processarem uma mesma cadeia, ambos chegam a estados finais, ou ambos chegam a estados não finais
- Para isto, são identificados estados não-equivalentes e, por exclusão, encontra-se os equivalentes
- Utiliza-se uma tabela triangular que possui um cruzamento para cada par de estados distintos do automato (incluindo S_{trash} quando este é acrescentado)
- Supondo um autômato com $S = \{S_0, S_1, \dots, S_{n-1}, S_n, S_{trash}\}$

imagem

 - * Estados não equivalentes serão sinalizados, ao final do processo, os cruzamentos em branco deverão indicar estados equivalentes.
- Inicia-se a marcação pelos estados trivialmente não-equivalentes
 - * Dois estados não trivialmente não-equivalentes se um é final e o outro não
- Exemplo:
 - * Considere o seguinte AFD e sua tabela de cruzamento de estados

imagem

...
- Critério usado para a marcação dos estados não equivalentes:
 - * Para cada par $\{S_u, S_v\}$ não marcado e para cada simbolo $a \in \Sigma$, suponha que $\delta(S_u, a) = R_u$ e $\delta(S_v, a) = R_v$
 - Se $R_u = R_v$, então S_u e S_v são equivalentes e, para o simbolo a não deve ser marcado
 - Se $R_u \neq R_v$, e o par $\{R_u, R_v\}$ não está marcado, então $\{S_u, S_v\}$ é incluído em uma lista a partir de $\{R_u, R_v\}$ para posterior analise
 - Se $R_u \neq R_v$, e o par $\{R_u, R_v\}$ está marcado, então:
 - $\{S_u, S_v\}$ é não equivalente e deve ser marcado
 - Se $\{S_u, S_v\}$ encabeça uma lista de pares, então todos os pares da lista devem ser marcados
- Na unificação dos estados equivalentes:
 - * Conjuntos de estados finais equivalentes podem ser unificados como um unico estado final

- * Conjuntos de estados não-finais equivalentes podem ser unificados como um unico estado não-final
- * Se algum dos estados equivalentes é inicial, então o correspondente estado unificado é inicial

4 Expressões Regulares

- Formalismo de carater gerador
 - A partir deee pode-se inferir como construir as cadeias da linguagem
 - Pode representar qualquer LR – Linguagem Regular
 - Uma ER é definida a partir de conjuntos básicos e operadores de concatenação e união
 - Formalismo adequado para comunicação homem $\times$ homem e homem $\times$ máquina
 - Uma ER sobre um alfabeto Σ é definida como segue:
 - $\emptyset$ é uma ER e denota a linguagem vazia
 - ε é uma ER e denota a linguagem contendo exclusivamente a palavra vazia, ou seja, $\{\varepsilon\}$
 - Qualquer simbolo x pertence a Σ é uma ER e denota a linguagem contendo a palavra x , ou seja, $\{x\}$
 - Se r e s são ERs e denotam as linguagens R e S , respectivamente, então:
 - * $(r + s)$ é ER e denota a linguagem $R \cup S$ (União)
 - * (rs) é ER e denota a linguagem $RS = \{uv | u \in R \text{ e } v \in S\}$ (Concatenação)
 - * (r^*) é ER e denota a linguagem R^* (Fecho)
 - Pode-se utilizar parenteses ou não, mas deve-se considerar a seguinte convenção:
 - * A concatenação sucessiva (fechamento ou fecho de Kleene) tem precedencia sobre a concatenação
 - * A concatenação tem precedência sobre a união
 - Exemplos de ER
- Tabela
- ERs podem ser usadas em buscas em editores de texto

5 Gramática

- Mecanismo gerador que permite definir formalmente uma linguagem
- Através de uma gramática pode-se gerar todas as sentenças da linguagem definida por ela
- Modelo muito aplicado na especificação de linguagens computacionais
- Formalmente, gramática é uma quadrupla $G = (V, T, P, S)$ onde:

- V é um conjunto finito de símbolos não-terminais (ou variáveis)
- T é um conjunto finito de símbolos terminais disjunto de V
- P é um conjunto finito de pares, denominados regras de produção tal que a primeira componente é uma palavra de $(V \cup T)^+$ e a segunda componente é uma palavra $(V \cup T)^*$
- S é um elemento de V , denominado símbolo inicial (ou símbolo de partida)
- Os símbolos de T equivalem aqueles que aparecem nos programas de uma linguagem de programação. É o alfabeto em cima do qual a linguagem é definida
- Os elementos de V são símbolos auxiliares que são criados para permitir a definição das regras da linguagem. Eles correspondem à "categorias sintáticas" da linguagem definida
 - Português: sentença, predicado, verbo, ...;
 - Pascal: programa, bloco, procedimento, ...;
- Uma regra de produção (α, β) é representada por $\alpha \rightarrow \beta$
- As regras de produção definem as condições de geração das sentenças
- A aplicação de uma regra de produção é denominada derivação
- Uma regra $\alpha \rightarrow \beta$ indica que α pode ser substituído por β sempre que α aparecer
- Enquanto houver símbolo não-terminal na cadeia em derivação, esta derivação não terá terminado
- O símbolo inicial é o símbolo através do qual deve iniciar o processo de derivação de uma sentença
- Observações:
 - $V \cap T = \emptyset$
 - Os elementos de T são os terminais. Procuraremos representá-los por letras minúsculas (a,b,c,d, ...)
 - Os elementos de V são os não-terminais. Procuraremos representá-los por letras maiúsculas (A,B,C,D, ...)
 - As cadeias mistas, isto é, aquelas que contém símbolos de V e símbolos de T (cadeias pertencentes à $(V \cup T)^*$) serão representadas por letras gregas ($\alpha, \beta, \gamma, \delta, \dots$)
- Exemplo:
 - $G_1 = (\{S, A, B\}, \{a, b\}, P, S)$ onde:
 - * $P = \{1)S \rightarrow AB$
 - 2) $A \rightarrow a$
 - 3) $B \rightarrow b\}$
 - Quais as cadeias terminais geradas por esta gramática?
 - R: ab

- Descrição de linguagens:

1. $G_1 = (\{A, B\}, \{a, b, c\}, P, A)$ onde:

$$P = \{1) A \rightarrow aB$$

$$2) B \rightarrow bB$$

$$3) B \rightarrow c\}$$

2. $G_2 = (\{S, A, B, C\}, \{a, b, c\}, P, S)$ onde:

$$P = \{1) S \rightarrow A$$

$$2) A \rightarrow BaC$$

$$3) B \rightarrow bB$$

$$4) B \rightarrow b$$

$$5) C \rightarrow cC$$

$$6) C \rightarrow c\}$$

– A linguagem gerada pela gramática G_1 descrita acima, poderia ser expressa da seguinte forma: $L(G_1) = \{ab^n c, n \geq 0\}$

– A linguagem gerada pela gramática G_2 descrita acima, poderia ser expressa da seguinte forma: $L(G_2) = \{b^m a c^n, m \geq 1, n \geq 1\}$

- Geração direta ($\Rightarrow$):

– Considere $\alpha, \beta, \gamma, \delta \in (V \cup T)^*$

– Uma cadeia $\alpha\gamma\beta$ gera diretamente ($\Rightarrow$) uma cadeia $\alpha\delta\beta$ se e somente se:

$$\gamma \rightarrow \delta \in P$$

- Geração ($\Rightarrow^*$):

– Uma cadeia α gera ($\Rightarrow^*$) uma cadeia β se e somente se:

$$\exists \gamma_1, \gamma_2, \dots, \gamma_n | \alpha \rightarrow \gamma_1 \rightarrow \gamma_2 \rightarrow \dots \rightarrow \gamma_n \rightarrow \beta, n \geq 0$$

- Definições:

– Forma sentencial: uma cadeia $\alpha \in (V \cup T)^*$ é uma forma sentencial de uma gramática se e somente se $S \Rightarrow^* \alpha$, ou seja, α é um "embrião" para alguma sentença gerada pela gramática ou a própria sentença

– Sentença: uma forma sentencial α , é uma sentença de G se e somente se $\alpha \in T^*$. Portanto, as cadeias terminais geradas pela gramática são as sentenças de G

- Tipos de Gramática

– A cada classe de linguagem da Hierarquia de Chomsky é associado um tipo de gramática

– Gramática com Estrutura de Frase – GEF ou Tipo 0

* São as gramáticas onde todas as regras de produção pertencentes ao conjunto P são da forma:

$$\alpha \rightarrow \beta \text{ onde } \beta \in (V \cup T)^* \wedge \alpha \in (V \cup T)^+$$

- * Os próximos tipos de gramática são GEFs com restrições
- Gramática Sensível ao Contexto – GSC ou Tipo 1
 - * São as gramáticas onde todas as regras de produção pertencentes ao conjunto P são da forma:

$$\alpha \rightarrow \beta \text{ tal que } |\beta| \geq |\alpha| \text{ exceto quando } \beta = \lambda$$

$$\beta \in (V \cup T)^*$$

$$\alpha \in (V \cup T)^+$$

- * Exemplo:

$$S \rightarrow aSBC$$

$$S \rightarrow aBC$$

$$CB \rightarrow BC$$

$$aB \rightarrow ab$$

$$bB \rightarrow bb$$

$$bC \rightarrow bc$$

$$cC \rightarrow cc$$

$$\text{Gera a linguagem: } L(G) = \{a^n b^n c^n | n \geq 0\}$$

- Gramática Livre de Contexto – GLC ou Tipo 2
 - * São as gramáticas onde todas as regras de produção pertencentes ao conjunto P são da forma:

$$A \rightarrow \beta \text{ onde}$$

$$\beta \in (V \cup T)^*$$

$$A \in V$$

- * Exemplos:

$$\cdot S \rightarrow AB$$

$$A \rightarrow 0A11$$

$$A \rightarrow \lambda$$

$$B \rightarrow 0B$$

$$B \rightarrow \lambda$$

$$\cdot S \rightarrow aB$$

$$S \rightarrow bA$$

$$A \rightarrow a$$

$$A \rightarrow aS$$

$$A \rightarrow bAA$$

$$B \rightarrow b$$

$$B \rightarrow bS$$

$$B \rightarrow aBB$$

- * O próximo tipo de gramática é GLC com restrições
- Gramática Regular – GR ou Tipo 3
 - * Uma linguagem regular é uma linguagem que pode ser descrita por uma gramática linear
 - * Tipos de gramática linear:
 - Gramática Linear à Direita – GLD

- Gramática Linear à Esquerda – GLE
- Gramática Linear Unitária à Direita – GLUD
- Gramática Linear Unitária à Esquerda – GLUE
- * Seja $G = (V, T, P, S)$ e sejam A e B elementos de V , e w uma cadeia de T^* , então:
 - GLD – $A \rightarrow wB \vee A \rightarrow w$
 - GLE – $A \rightarrow Bw \vee A \rightarrow w$
 - GLUD – $A \rightarrow wB \vee (A \rightarrow w \wedge |w| \leq 1)$
 - GLUE – $A \rightarrow Bw \vee (A \rightarrow w \wedge |w| \leq 1)$

6 Algoritmo de reconhecimento para LLC

- Algoritmo de Cocke-Younger-Kasami
 - Desenvolvido independentemente por Cocke, Younger e Kasami em 1965
 - Trabalha sobre uma GLC na Forma Normal de Chomsky (pré-requisito)
 - * Portanto, o algoritmo atende LLCs que não possuem a palavra vazia
 - Realiza uma análise ascendente (bottom-up) das cadeias
 - O algoritmo parte da cadeia a ser analisada e regride nas produções formando uma tabela triangular
 - Quando o triângulo estiver completo, a cadeia é aceita se o símbolo de partida da gramática estiver no topo do mesmo
 - O estado inicial precisa estar no topo da tabela
 - Considerando as dimensões "r" e "s" da tabela, conforme ilustra a figura a seguir:
 1. Regressão dos terminais
 2. Regressão dos não terminais

7 Autômato com Pilha

- São formalismos (máquinas) capazes de reconhecer as Linguagens Livres de Contexto
- Maior poder que os Autômatos Finitos, pois possuem "um espaço de armazenamento" extra que é utilizado durante o processamento de uma cadeia
- Possui uma pilha que caracteriza uma memória auxiliar onde pode-se inserir e remover informações
- Mesmo poder de reconhecimento das GLCs
- Um exemplo de LLC: $\{a^n b^n | n \geq 0\}$
- Um AF não é capaz de reconhecer este tipo de linguagem devido à sua incapacidade de "recordar" (memorizar) informação sobre a cadeia analisada
- Autômatos com Pilha (AP) possuem uma pilha para armazenar informação, adicionando poder aos AFs

- Definição:
 - AP é uma sextupla $\langle \Sigma, \Gamma, S, S_0, \delta, B \rangle$, onde:
 - * Σ é o alfabeto de entrada do AP;
 - * Γ é o alfabeto da pilha;
 - * S é o conjunto finito não vazio de estados do AP;
 - * S_0 é o estado inicial, $S_0 \in S$;
 - * δ é a função de transição de estados, $\delta : S \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow$ Conjunto de subconjuntos finitos de $S \times \Gamma^*$
 - * B é o simbolo da base da pilha, $B \in \Gamma$
- Ao contrário da fita de entrada, a pilha pode ser lida e alterada durante um processamento
- O autômato verifica o conteúdo do topo da pilha, retira-o e substitui por uma cadeia $\alpha \in \Gamma^*$.
 - Se $\alpha = A$, e $A \in \text{Gamma}$, então o simbolo do topo é substituido por A e a cabeça de leitura escrita continua posicionada no mesmo lugar
 - Se $\alpha = A_1 A_2 \dots A_n, n > 1$ então o simbolo do topo da pilha é retirado, sendo A_n , colocado em seu lugar A_{n-1} na posição seguinte, e assim por diante. A cabeça é deslocada para a posição ocupada por A_1 que é então o novo topo da pilha
 - Se $\alpha = \lambda$ então o símbolo do topo da pilha é retirado, fazendo a pilha decrescer
- A função de transição δ , é função do estado corrente, da letra corrente na fita de entrada e do símbolo no topo da pilha.
- Além disso, esta função determina não só o próximo estado que o AP assume, mas também como o topo da pilha deve ser substituído.
- O AP inicia sua operação num estado inicial especial denotado por S_0 e com um único símbolo na pilha, denotado por B .
- A configuração de um AP é dada por uma tripla $\langle s, x, \alpha \rangle$ onde s é o estado corrente, x é a cadeia da fita que falta ser processada e α é o conteúdo da pilha, com o topo no inicio de α
- O AP anda ou move-se de uma configuração para outra através da aplicação de uma função de transição.
- Se o AP está na configuração $\langle s, ay, A\beta \rangle$ e temos que $\delta(s, a, A) = \langle t, \gamma \rangle$ e denota-se $\langle s, ay, A\beta \rangle \vdash \langle t, y, \gamma\beta \rangle$.
- Se o AP move-se de uma configuração $\langle s_1, x_1, \alpha_1 \rangle$ para uma configuração $\langle s_2, x_2, \alpha_2 \rangle$ por meio de um número finitos de movimentos, denotamos:

$$\langle s_1, x_1, \alpha_1 \rangle \vdash^* \langle s_2, x_2, \alpha_2 \rangle$$

OBS.: $\vdash \Rightarrow, \vdash^* \Rightarrow^*$

- Se o valor de δ para uma determinada configuração for $\emptyset$ o AP para.
- Note que, segundo esta definição, APs não possuem estados finais como os AFs
- Assim, uma cadeia x é aceita se, ao chegar no final do processamento da mesma, a pilha estiver vazia, independentemente do estado que em o AP se encontra
- Formalmente temos que:
 - Dado o AP $P = \langle \Sigma, \Gamma, S, S_0, \delta, B \rangle$ e a cadeia x sobre Σ , diz-se que x é aceita por P se e somente se $\exists s \in S$ tal que $\langle S_0, x, B \rangle \vdash^* \langle s, \gamma, \gamma \rangle$. Caso contrário, x é rejeitada.
 - Dado o AP $P = \langle \Sigma, \Gamma, S, S_0, \delta, B \rangle$, a linguagem $L(P)$ definida por P é:

$$\{x \in \Sigma^* | \exists s \in S \exists \langle S_0, x, B \rangle \vdash^* \langle s, \gamma, \gamma \rangle\}$$

8 Máquina de Turing

- Introduzida por Alan Turing em 1936
- Ferramenta para estudar a capacidade dos processos algorítmicos
- Modelo abstrato, concebido antes mesmo de uma implementação tecnológica
- Formaliza a ideia de uma pessoa que realiza cálculos
 - Simulação de uma situação na qual uma pessoa, equipada com um instrumento de escrita e um apagador, realiza cálculos numa folha de papel
 - "A Máquina de Turing ainda hoje é aceita como a formalização de um procedimento efetivo, ou seja, uma sequência finita de instruções, as quais podem ser realizadas mecanicamente, em tempo finito
- MT pode ser vista como a mais simples "máquina de computação", servindo para determinar quais funções são computáveis e quais não são.
- Outros modelos foram propostos, mas todos mostraram ter, no máximo, poder computacional equivalente ao da MT
- Estas são chamadas Máquinas Universais:
 - Máquinas capazes de expressar a solução para qualquer problema algorítmico
- A Máquina de Turing consiste de:
 - Uma Unidade de Controle
 - * Pode ler e escrever símbolos em uma fita por meio de um cabeçote de leitura e gravação e pode se deslocar para a esquerda ou direita
 - A fita estende-se infinitamente em uma das extremidades e é dividida em células. Utilizada como dispositivo de entrada, saída e memória de trabalho
 - * Estas células podem armazenar qualquer elemento de um conjunto finito de símbolos, um alfabeto

- Programa ou Função de Transição: função que comanda as leituras e gravações, o sentido de movimento da cabeça e define o estado da máquina que será ativado imagem
- Funcionamento da Máquina de Turing
 - A MT deve assumir sempre um estado, pertencente à um conjunto finito de estados
 - O processamento de uma MT começa sempre em um estado especial, chamado estado inicial
 - Inicialmente a primeira célula da fita é preenchida com " $\langle$ ", que indica o início da mesma
 - A cabeça de leitura é posicionada inicialmente na segunda célula da fita, a célula seguinte a " $\langle$ "
 - As células em branco, que não fazem parte da palavra a ser processada, são preenchidas com o símbolo " β "
 - O processamento em uma MT consiste em:
 - * Observar o estado e o símbolo corrente da fita (aquele em que o cabeçote está posicionado)
 - * Escrever um símbolo nesta célula da fita
 - * Mover o cabeçote para a esquerda ou direita
 - * Definir o estado corrente
 - A ação exata a ser executada é determinada por um programa (função de transição) que comunica à unidade de controle o que deve ser feito com base na configuração (estado + símbolo corrente da fita) em que a MT se encontra
 - O processamento cessa quando a máquina atinge uma configuração para a qual não existe função prevista, neste caso
 - * Se a máquina estiver com um estado final ativo, a cadeia de entrada é aceita
 - * Se o estado ativo não for final, a cadeia de entrada não é aceita
 - * Se a máquina não parar (looping), a cadeia de entrada não é aceita
- Definição da MT, uma óctupla:
 - Σ : alfabeto de símbolos de entrada
 - S : conjunto de estados possíveis, o qual é finito
 - δ : programa ou função de transição
$$\delta : S \times (\Sigma \cup V \cup \{\beta, \langle\}) \rightarrow S \times (\Sigma \cup V \cup \{\beta, \langle\}) \times \{E, D\}$$
 - s_0 : estado inicial da máquina, $s_0 \in S$
 - F : conjunto de estados finais, $F \subset S$
 - V : alfabeto auxiliar (pode ser vazio)
 - β : símbolo especial para células em branco
 - $\langle$: símbolo especial marcador de início da fita

- A Máquina de Turing pode ser empregada como modelo de natureza reconhecedora ou transdutora
 - Reconhecedora: processa a palavra de entrada aceitando-a ou rejeitando-a. Neste caso, o conjunto de palavras aceitas corresponde à linguagem descrita pela MT
 - Transdutora: máquina para computar uma função. Aplica uma função sobre o conteúdo inicial da fita e o resultado produzido é lançado na própria fita
- Representação de MT por diagrama:
 - Semelhante à representação de AFD
 - Os rótulos das setas, que representam as funções de transição são de acordo com a seguinte legenda:
imagem
- Configuração instantânea de uma MT:
 - Seja uma MT $M = (\Sigma, S, \delta, s_0, F, V, \beta, \langle \rangle)$
 - * A configuração instantânea é dada por um par $[e, x\underline{a}y]$, em que:
 - $e \in S$ é o estado atual (ou ativo)
 - $x \in \{\Sigma \cup V \cup \{\beta, \langle \rangle\}^*$ é a palavra situada à esquerda da cabeça de leitura
 - $a \in \{\Sigma \cup V \cup \{\beta, \langle \rangle\}$ é o símbolo que se encontra sob a cabeça de leitura;
 - e
 - $y \in \{\Sigma \cup V \cup \{\beta\}^*\}$ é a palavra à direita da cabeça de leitura até o último símbolo diferente de β ; se não existir símbolo diferente de β , $y = \lambda$
 - Com isto, estabelece-se a relação $\vdash$ ("leva a")
 - No exemplo descrito, a função $\delta(S_0, 1) = (S_0, 0, D)$ levou a máquina da configuração $[S_0, \langle \underline{1}010]$ para a configuração $[S_0, \langle 0\underline{0}10]$, assim pode-se escrever:
 - * $[S_0, \langle \underline{1}010] \vdash [S_0, \langle 0\underline{0}10]$
 - Adicionalmente, pode-se estender as notações:
 - * $\vdash^n$: quando através da aplicação de um número n (conhecido) de funções uma configuração leva a outra
 - $[S_0, \langle \underline{1}010] \vdash^1 [S_0, \langle 0\underline{0}10]$
 - $[S_0, \langle \underline{1}010] \vdash^{10} [S_f, \langle \underline{0}101]$
 - * $\vdash^*$: quando através da aplicação n funções (com $n \geq 0$) uma configuração pode leva a outra
- Considerações acerca da aceitação ou não de uma entrada:
 - Considerando que o modelo de MT definido é determinístico, para uma dada entrada a MT deve parar sempre em um mesmo estado e
 - Com isto, se e é estado final, a cadeia é aceita, caso contrário ela é rejeitada
 - Adicionalmenrem a MT pode não parar (entrar em looping). Neste caso, a entrada é rejeitada

- Particularidades das MTs:
 - Ao contrário de AFs e APs, não é necessário que a palavra de entrada seja toda lida para que possa ser aceita ou rejeitada
 - A classe de linguagem que MT é capaz de reconhecer é a classe das Linguagens Enumeráveis Recursivamente
 - Esta classe contém as linguagens regulares e livres de contexto como subclasses (conforme hierarquia de Chomsky)
 - Assim, toda linguagem que um AF ou AP reconhece, pode ser reconhecida por uma MT
 - Dada um LER, existe uma MT que para para qualquer entrada desta linguagem? Não!
 - Daí definiu-se uma subclasse para LERs, a classe das linguagens recursivas:
 - * Uma linguagem L é dita recursiva se existe uma MT que reconhece todas as palavras de L e para
- Com a inclusão das linguagens recursivas, o diagrama da hierarquia de Chomsky poderia ser descrito da seguinte forma:

imagem

- Classes de problemas:

imagem

- Exemplo de problema insolúvel:
 - * Problema da Parada (Halting Problem)
 - Descrição: O Problema da Parada pergunta se, dado um programa P e uma entrada x , o programa P vai parar (ou seja, terminar a execução) ou continuar rodando indefinidamente quando é executado com a entrada x
 - Insolubilidade: Alan Turing provou que não existe um algoritmo real que resolva o Problema da Parada para todos os programas e todas as entradas possíveis. Isso significa que não existe uma máquina de Turing capaz de determinar, em todos os casos, se um programa vai parar ou não
 - Intuição: A ideia é que, para alguns programas, tentar determinar se eles param pode levar a contradições lógicas, tornando o problema indecidível. Portanto, o Problema da Parada é um exemplo de um problema computacional que é insolúvel
 - * Problema do Caixeiro Viajante
 - Encontrar o caminho mais curto que visita todos os pontos em um conjunto dado de pontos e retorna ao ponto de origem. Embora existam algoritmos que podem encontrar soluções aproximadas, encontrar a solução exata para grandes conjuntos de pontos é computacionalmente inviável em um tempo razoável

- Formalismo reconhecedor para LSC:

- Uma linguagem é sensível ao contexto quando:
 - * Pode ser gerada por uma GSC; ou
 - * Pode ser reconhecida por um Autômato Linearmente Limitado (ALL) ou Máquina de Turing com Fita Limitada
- Um ALL é como a definição de MT estudada com a inclusão de :
 - * Um símbolo especial " ϵ " usado para marcar o final da fita
 - * Possibilidade de não-determinismo na criação das funções
- Máquina de Turing
 - Dada a sua natureza conceitual, a MT pode ser implementada de diversas formas
 - Os computadores modernos são MT (exceto pelo fato de terem memória finita)
 - * O processador corresponde à unidade de controle, cujos estados podem ser definidos pelos padrões de bits que podem ser associados aos registradores
 - * A memória da máquina corresponde ao sistema de armazenamento em fita
 - * Os padrões de bits (0 e 1) correspondem ao alfabeto da fita
 - A importância da MT para a Ciência da Computação:
 - * A potência computacional da MT é tão grande quanto a de qualquer sistema algorítmico
 - * Se um problema não puder ser resolvido por uma MT, não poderá ser resolvido por qualquer sistema algorítmico
 - * MT representa a fronteira teórica da capacidade computacional para as máquinas modernas reais